

STM32Cube小记 (01): 输入捕获 + DMA 实现信号频率与占空比的测量

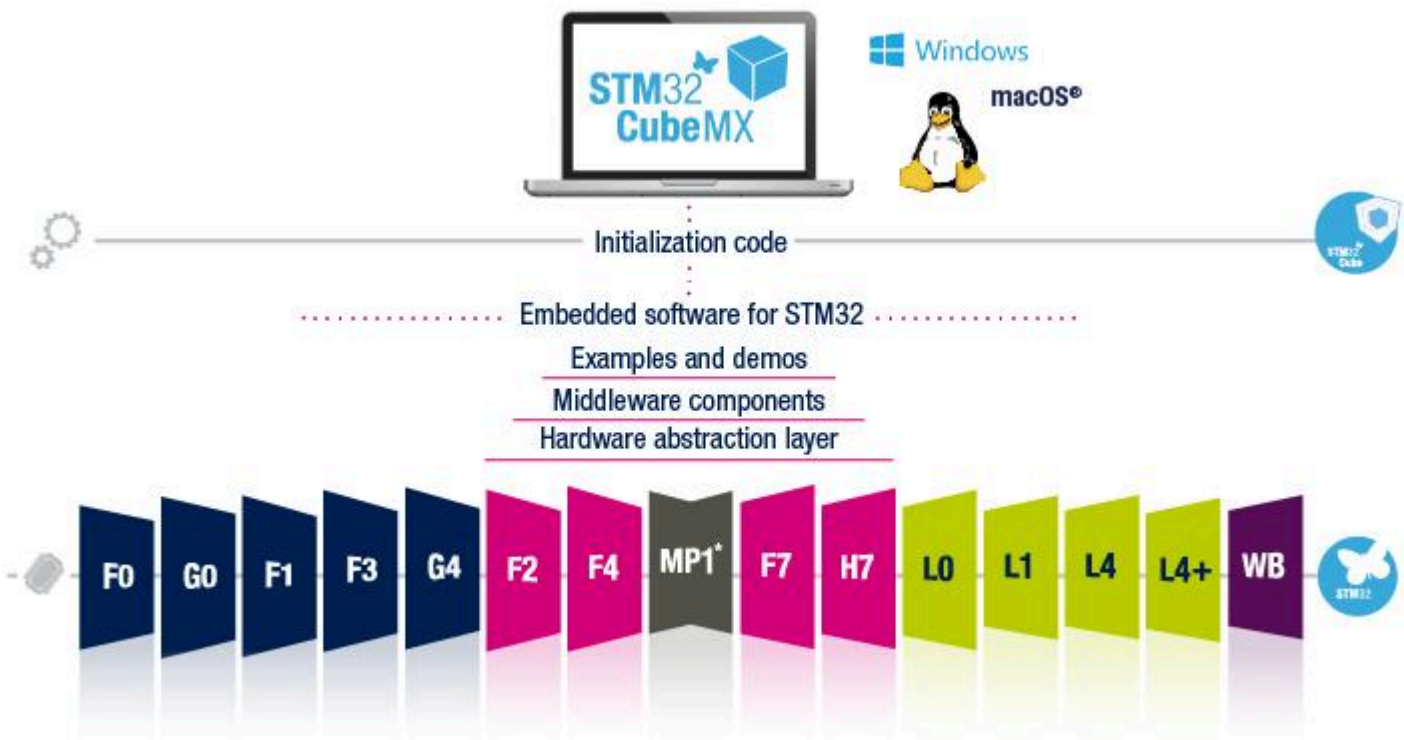
由 CNPP
ID:476

https://www.emoe.xyz/stm32_dma_capture/

July 29, 2019

分类目录: 嵌入式, 所有文章, 硬件

标签: STM32, 项目



本文目录

STM32Cube小记(01): 输入捕获 + DMA 实现信号频率与占空比的测量

前言

CubeMX 配置

函数编写

运行效果

后记 + 吐槽

吐槽1

STM32Cube小记(01): 输入捕获 + DMA 实现信号频率与占空比的测量

作者	日期	硬件工具	软件工具
CNPP	2019-07-29	NUCLEO-L432KC	STM32CubeMX, MDK5

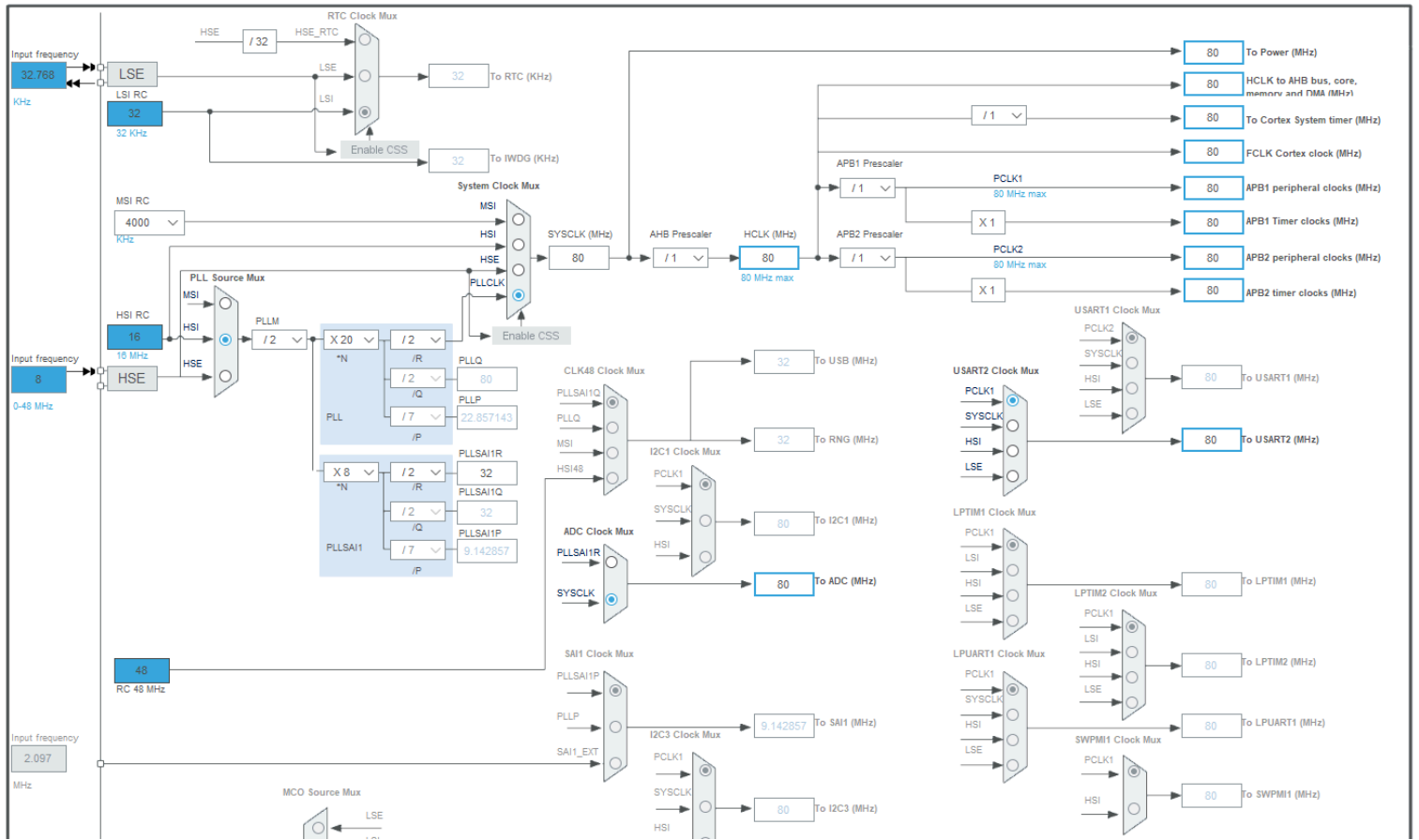
前言

此前笔者曾尝试过使用CubeMX配置STM32普通定时器的PWM输入模式，设定CH1为上升沿捕获，CH2为下降沿捕获，通过在中断回调函数中读取并处理CCR1、CCR2寄存器的值，获得被测信号的频率及占空比。PWM输入模式的具体操作可参考STM32CubeMX中附带的针对各官方板卡的例程，笔者使用的板卡为NUCLEO-L432KC，故对应的例程位于CubeMX安装目录下的 **Repository\STM32Cube_FW_L4_V1.14.0\Projects\NUCLEO-L432KC\Examples\TIM\TIM_PWMInput**中。

使用输入捕获 + 中断的方式，其优点在于操作简单，不阻塞主函数，但缺点也十分明显，即当所测信号频率较高时，单片机会反复进入输入捕获中断，极大地影响了其他程序的运行。所以，我们需要一种尽可能不需要进入中断的处理方法。结合使用DMA循环传输ADC采样值的例子，笔者自然也希望能使用DMA来实现真正非阻塞的输入捕获测频。

CubeMX 配置

首先照例配置时钟，笔者手上这块L432的好处就是各总线时钟及总线外设时钟都能配置为最高的80MHz，不必费心去查定时器挂在那条总线上了（偷懒），如果你使用的也是NUCLEO-32板，建议也像笔者这样直接使用内部HSI RC（具体原因可见本文末的吐槽2）。



接下来配置用于输入捕获的定时器，这里使用的是32位定时器TIM2，相较于其他16位定时器，其更不易溢出，故更适合采集低频信号。TIM2的总线时钟为80MHz，预分频8，故所能测量的最低频率为 $(80000000 / (8 * 4294967295))$ ，约为0.002Hz。

选择CH1为信号输入端，CH1为直连模式，CH2为非直连模式，这样CH2会在单片机内部与CH1相连，从而节省一个管脚。然后将定时器从模式（Slave Mode）设为Reset，并设置TL1FP1为触发源，这样，当CH1被触发时将复位TIM2所有的计数器，并更新CCR寄存器。

Mode

Slave Mode	Reset Mode	▼
Trigger Source	TI1FP1	▼
Clock Source	Disable	▼
Channel1	Input Capture direct mode	▼
Channel2	Input Capture indirect mode	▼
Channel3	Disable	▼
Channel4	Disable	▼
Combined Channels	Disable	▼
Use ETR as Clearing Source	Disable	▼
☐ XOR activation		
☐ One Pulse Mode		

然后配置预分频寄存器，使TIM2使用10MHz信号源，自动装填寄存器预装载值设为最大，即0xFFFFFFFF（对于16位定时器则是0xFFFF），然后将CH1配置为上升沿触发，CH2配置为下降沿触发。

（其实，若不需要开启DMA，可以直接在Combine Channels中开启PWM Input模式，这样更加方便，但在此模式下CubeMX在DMA选项页只会给出CH1的DMA请求选项。）

Configuration

Reset Configuration

Parameter Settings

User Constants

NVIC Settings

DMA Settings

GPIO Settings

Configure the below parameters :

Search (Ctrl+F)

Counter Settings

Prescaler (PSC - 16 bits value)

Counter Mode

Counter Period (AutoReload Register - 32 bits value)

Internal Clock Division (CKD)

auto-reload preload

Slave Mode Controller

8-1

Up

0xFFFFFFFF

No Division

Disable

Reset Mode

Trigger Output (TRGO) Parameters

Master/Slave Mode (MSM bit)

Trigger Event Selection TRGO

Disable (Trigger input effect not delayed)

Reset (UG bit from TIMx_EGR)

Input Capture Channel 1

Polarity Selection

IC Selection

Prescaler Division Ratio

Input Filter (4 bits value)

Rising Edge

Direct

No division

0

Input Capture Channel 2

Polarity Selection

IC Selection

Prescaler Division Ratio

Falling Edge

Indirect

No division

DMA配置，两通道均为循环模式，外设、内存数据宽度均为Word。之后配置好串口等其他外设，生成工程。

Configuration

Reset Configuration

Parameter Settings

User Constants

NVIC Settings

DMA Settings

GPIO Settings

DMA Request	Channel	Direction	Priority
TIM2_CH1	DMA1 Channel 5	Peripheral To Memory	Medium
TIM2_CH2/CH4	DMA1 Channel 7	Peripheral To Memory	Medium

Add

Delete

DMA Request Settings

Mode

Circular

Increment Address

Data Width

Word

Peripheral

Memory

函数编写

为了方便，编写了相关函数，开启DMA传输后，只需运行**Freq_Calc**,即可将当前所测信号的频率和占空比分别保存在全局变量**Freq_Capture_Freq**和**Freq_Capture_Duty**中。

freq_meter.h

```
#ifndef __FREQ_METER_H
#define __FREQ_METER_H

#include "main.h"
#include "tim.h"

#define TIM2_Clock 80000000
#define TIM2_PSC 8 - 1
#define TIM2_ARR 0xFFFFFFFF

#define IC_BufSize 32 //DMA缓冲区大小

extern float Freq_Capture_Freq;
extern float Freq_Capture_Duty;

void Freq_Meter_Init(void);
void Freq_Meter_DeInit(void);
void Freq_Calc(void);

#endif
```

freq_meter.c

```
#include "freq_meter.h"
```

```
uint32_t Capture_Raise = {0x00000000}; //CH1的DMA缓冲区，用于存放上升沿捕获数据
```

```
uint32_t Capture_Fall = {0x00000000}; //CH2的DMA缓冲区，用于存放下降沿捕获数据
```

```
uint32_t Avg_Raise = 0x00000000;
```

```
uint32_t Avg_Fall = 0x00000000;
```

```
float Freq_Capture_Freq = 0.0; //所测信号频率
```

```
float Freq_Capture_Duty = 0.0; //所测信号占空比
```

```
void Freq_Meter_Init(void)
```

```
{
```

```
    //开始TIM2CH1的DMA传输
```

```
    HAL_TIM_IC_Start_DMA(&htim2, TIM_CHANNEL_1, Capture_Raise, IC_BufSize);
```

```
    //还原TIM2状态，然后开始TIM2CH2的DMA传输
```

```
    htim2.State = HAL_TIM_STATE_READY;
```

```
    HAL_TIM_IC_Start_DMA(&htim2, TIM_CHANNEL_2, Capture_Fall, IC_BufSize);
```

```
}
```

```
void Freq_Meter_DeInit(void)
```

```
{
```

```
    HAL_TIM_IC_Stop_DMA(&htim2, TIM_CHANNEL_1);
```

```
    HAL_TIM_IC_Stop_DMA(&htim2, TIM_CHANNEL_2);
```

```
}
```

```
void Freq_Calc(void)
```

```
{
```

```
    Avg_Raise = 0;
```

```
    Avg_Fall = 0;
```

```
    //取平均值
```

```
    for(uint16_t i = 0; i < IC_BufSize; i++)
```

```
    {
```

```
        Avg_Raise += Capture_Raise;
```

```
        Avg_Fall += Capture_Fall;
```

```
    }
```

```
    Avg_Raise /= IC_BufSize;
```

```
    Avg_Fall /= IC_BufSize;
```

```
    //计算频率及占空比
```

```
    if(Avg_Raise != 0)
```

```
    {
```

```

    Freq_Capture_Freq = (float)TIM2_Clock / (float)((TIM2_PSC + 1) * Avg_Raise);
    Freq_Capture_Duty = (float)Avg_Fall / (float)Avg_Raise;
}
}

```

需要注意的是，在开启TIM2CH2的DMA之前（即第二次对TIM2调用**HAL_TIM_IC_Start_DMA**之前），需要将**htim2.State**置为**HAL_TIM_STATE_READY**，因为在第一次调用函数**HAL_TIM_IC_Start_DMA**中会进行如下操作：

```

2076 else if ((htim->State == HAL_TIM_STATE_READY))
2077 {
2078     if ((pData == NULL) && (Length > 0U))
2079     {
2080         return HAL_ERROR;
2081     }
2082     else
2083     {
2084         htim->State = HAL_TIM_STATE_BUSY;
2085     }
2086 }

```

从而将**htim2.State**置为**HAL_TIM_STATE_BUSY**，而第二次调用**HAL_TIM_IC_Start_DMA**时则会进行如下判断

```

2064 HAL_StatusTypeDef HAL_TIM_IC_Start_DMA(TIM_HandleTypeDef *htim, uint32_t Channel, uint32_t *pData, uint16_t Length)
2065 {
2066     uint32_t tmpsmcr;
2067
2068     /* Check the parameters */
2069     assert_param(IS_TIM_CCX_INSTANCE(htim->Instance, Channel));
2070     assert_param(IS_TIM_DMA_CC_INSTANCE(htim->Instance));
2071
2072     if ((htim->State == HAL_TIM_STATE_BUSY))
2073     {
2074         return HAL_BUSY;
2075     }

```

从而直接跳过开启DMA及使能CH2的代码，这一点要特别注意。（至于笔者在这里踩坑的经过，可见[吐槽1](#)）

运行效果

设置好串口重定向，在主函数中添加如下代码，即通过串口打印频率及占空比（注：这里在两句printf前忘加了计算函数Freq_Calc()，大家请加上）：

```

117     Freq_Meter_Init();
118     /* USER CODE END 2 */
119
120     /* Infinite loop */
121     /* USER CODE BEGIN WHILE */
122     while (1)
123     {
124         printf("Freq:%0.2fHz\n", Freq_Capture_Freq);
125         printf("Duty:%0.2f%%\n\n", Freq_Capture_Duty * 100);

```

通过串口助手接收数据：



串口号：

COM12  ▼

波特率：

38400 ▼

数据位：

8 ▼

校验位：

None ▼

停止位：

One ▼

关闭串口

接收区设置.

☐ 接收并保存到文件

☐ 十六进制显示

☐ 暂停接收显示

☐ 自动断帧

?

20

☐ 接收脚本

Add Timest ▼

[保存数据](#) [清空数据](#)

发送区设置.

☐ 发送文件 [扩展命令](#)

☐ 十六进制发送

☐ 发送脚本

ADD8 ▼

☐ 定时发送

1.0 秒

☐ DTR ☐ RTS

Duty:57.14%

Freq:285714.28Hz
Duty:57.14%

Freq:285714.28Hz
Duty:57.14%

Freq:285714.28Hz
Duty:57.14%

Freq:285714.28Hz
Duty:57.14%

Freq:294117.66Hz
Duty:58.82%

Freq:285714.28Hz
Duty:57.14%

Freq:285714.28Hz
Duty:57.14%



发送：0

接收：6467

[复位计数](#)



串口号：

COM12

✓

▼

波特率：

38400

▼

数据位：

8

▼

校验位：

None

▼

停止位：

One

▼

关闭串口

接收区设置.

☐ 接收并保存到文件

☐ 十六进制显示

☐ 暂停接收显示

☐ 自动断帧

?

20

☐ 接收脚本

Add Timest

▼

保存数据

清空数据

发送区设置.

☐ 发送文件

扩展命令

☐ 十六进制发送

☐ 发送脚本

ADD8

▼

☐ 定时发送

1.0

秒

☐ DTR ☐ RTS

Duty:58.73%

Freq:5.04Hz

Duty:58.74%

Freq:5.04Hz

Duty:58.73%

Freq:5.04Hz

Duty:58.76%

Freq:5.04Hz

Duty:58.77%

Freq:5.04Hz

Duty:58.76%

Freq:5.04Hz

Duty:58.76%

Freq:5.04Hz

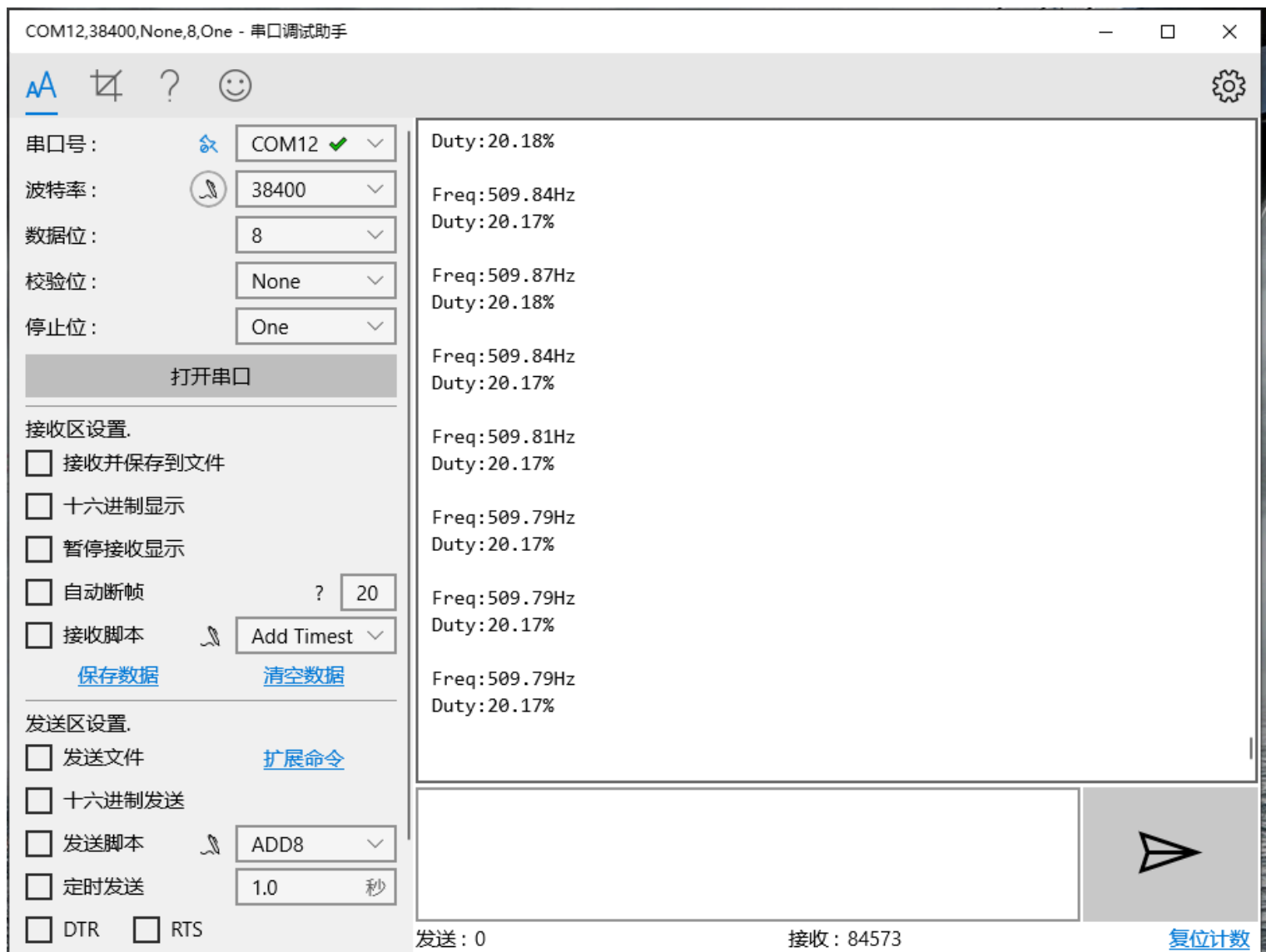
Duty:58.74%

发送：0

接收：8533

[复位计数](#)





后记 + 吐槽

吐槽1

其实只是一个挺简单的东西，到头来还是踩了不少坑2333。起初是想给正在尝试中的简易示波器加个频率计的，结果被输入捕获中断阻塞到连屏幕都无法刷新，于是就想寻找使用DMA的方案。其实很多红外解码的例子使用的就是输入捕获 + DMA，但为一个TIM的两个输入捕获通道开辟DMA请求的，鄙人着实没有找到例子，只能自己开干。起初初始化的代码是这样的：

```
void Freq_Meter_Init(void)
{
    HAL_TIM_IC_Start_DMA(&htim2, TIM_CHANNEL_1, Capture_Raise, IC_BufSize);
    HAL_TIM_IC_Start_DMA(&htim2, TIM_CHANNEL_2, Capture_Fall, IC_BufSize);
}
```

结果缓存根本收不到数据，仔细一看CCR2更是空空如也——CH2就没打开。后来，把第二句改成了**HAL_TIM_IC_Start**，这下CH2正常工作了，但DMA总归还是要有的，于是又查了半天，甚至怀疑到这个CH2/CH4上来（“是不是我打开的姿势不对”）：

Configuration

Reset Configuration

Parameter Settings

User Constants

NVIC Settings

DMA Settings

GPIO Settings

DMA Request	Channel	Direction	Priority
TIM2_CH1	DMA1 Channel 5	Peripheral To Memory	Medium
TIM2_CH2/CH4	DMA1 Channel 7	Peripheral To Memory	Medium

Add

Delete

DMA Request Settings

Mode

Circular

Increment Address

☐

Data Width

Word

Peripheral

☐

Memory

☒

Word

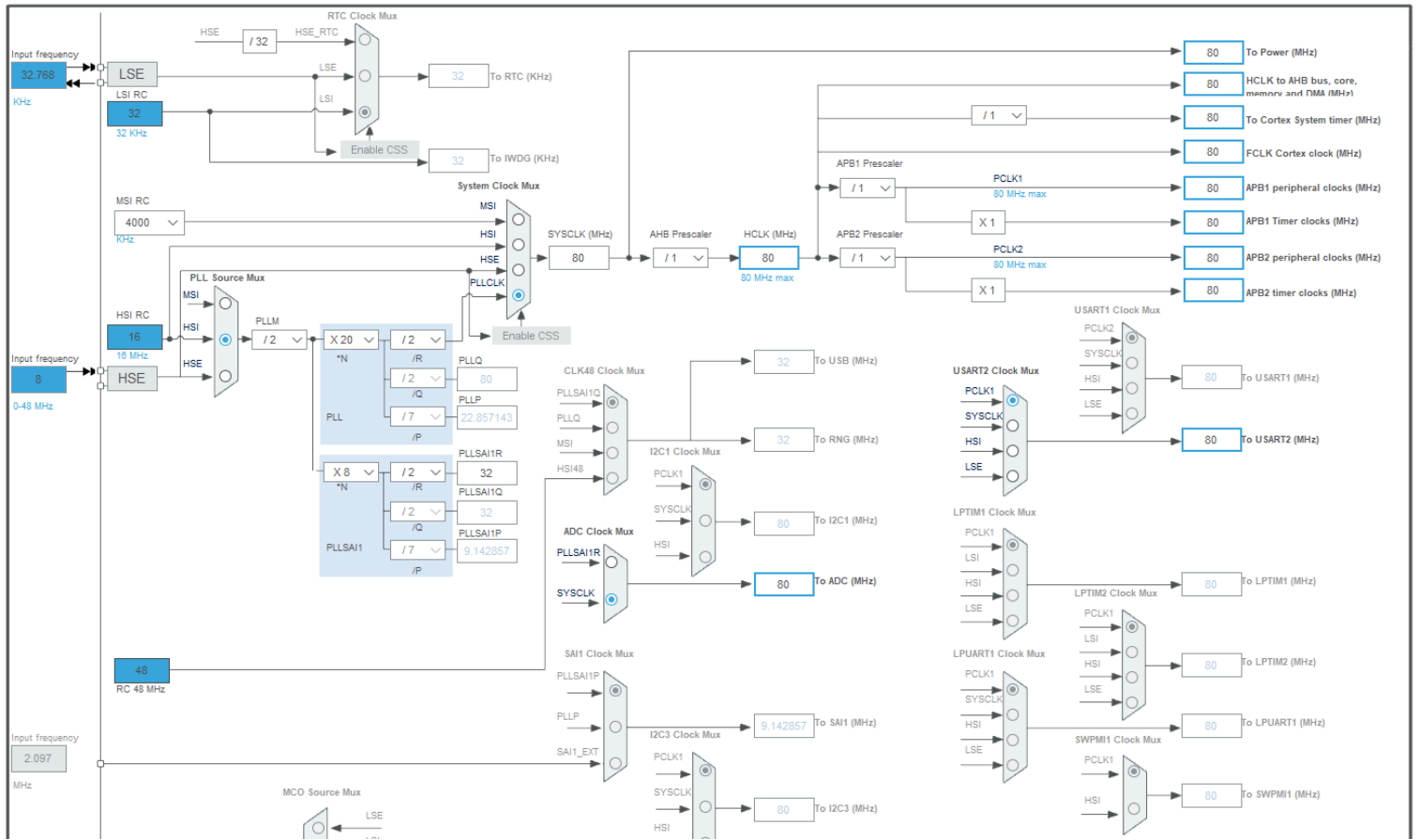
Word

最后，又重新比较了一遍HAL_TIM_IC_Start_DMA和HAL_TIM_IC_Start的区别，再加上断点调试，才发现了htim2.State的问题（调试指针从函数开头直接跳到结尾的那一刻，那感觉真是……）。总之教训就是，代码出Bug先从单步调试走起（逃）。

吐槽2

总算把DMA配好之后，就开始处理频率不对的问题（算出的频率是实际值的20倍2333）（其实早有察觉）。时钟树配置、定时器配置、甚至浮点数计算一波查下来，什么毛病都没有，我都开始怀疑是不是ST在暗中克扣总线外设速度了。无奈又用TIM2输出了一路PWM，结果不出所料，输出的频率是理论计算值的1/20……

无意间又瞟了一眼时钟树，突然意识到大事不妙：



首先我之前照例开启了CSS（Clock Securty System），简直就像给自己立Flag。而后L432KC的封装太小，高速时钟只能由外部提供（没法接无源晶振），在NUCLEO上应该是由STLink的MCO提供。再用示波器挂上PA0（高速时钟输入），结果根本就是一根直线。不管那么多，先切成内部RC，重配一遍时钟树，结果一下子就变正常了！连屏幕刷新也不知快了多少（我就纳闷为什么这SPI刷屏速度怎么搞的和I2C一样）（大哭）。

再翻来NUCLEO-32的手册，MCO这里果然有一个跳线，再翻过板子一看，跳线果然没连……

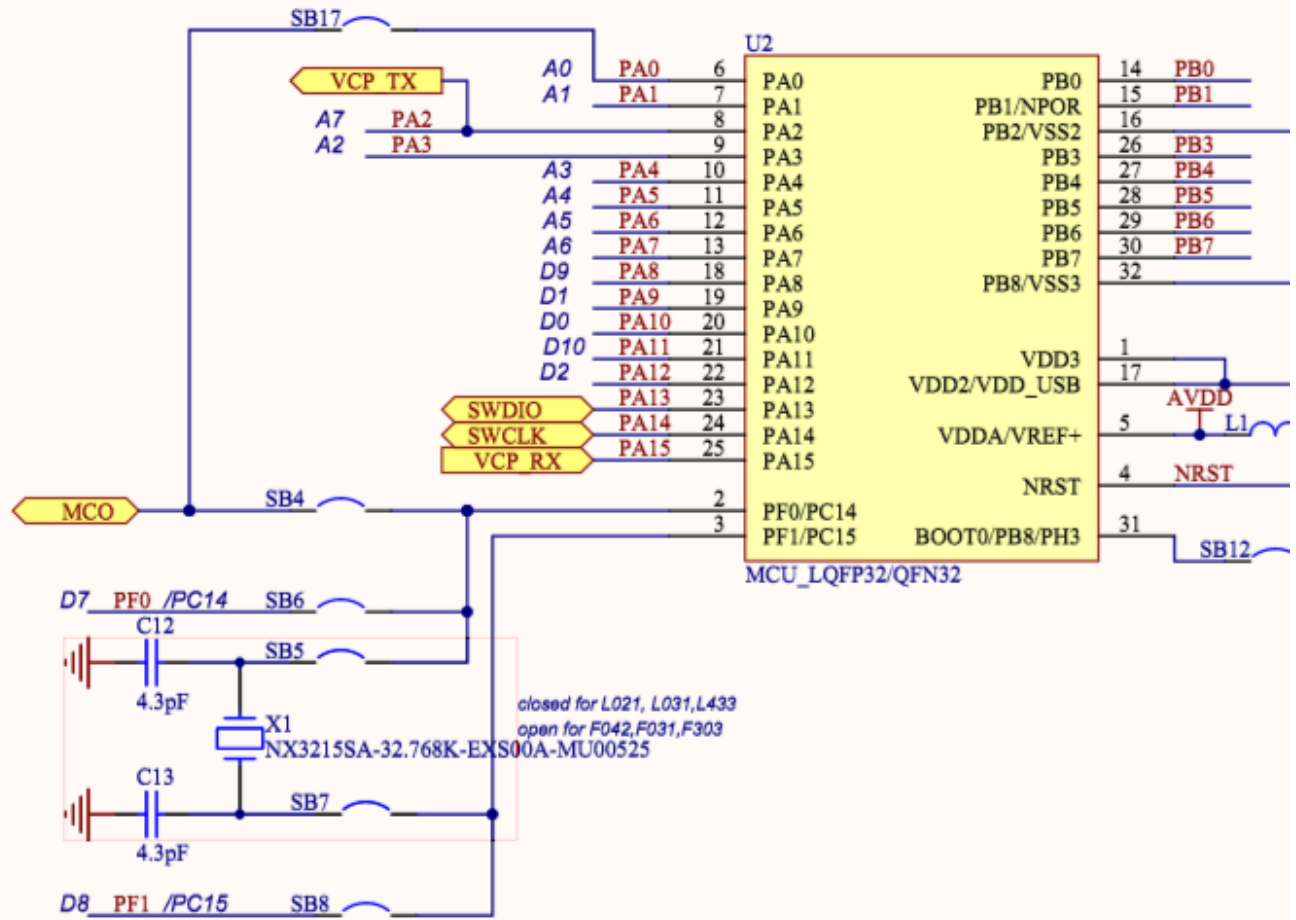
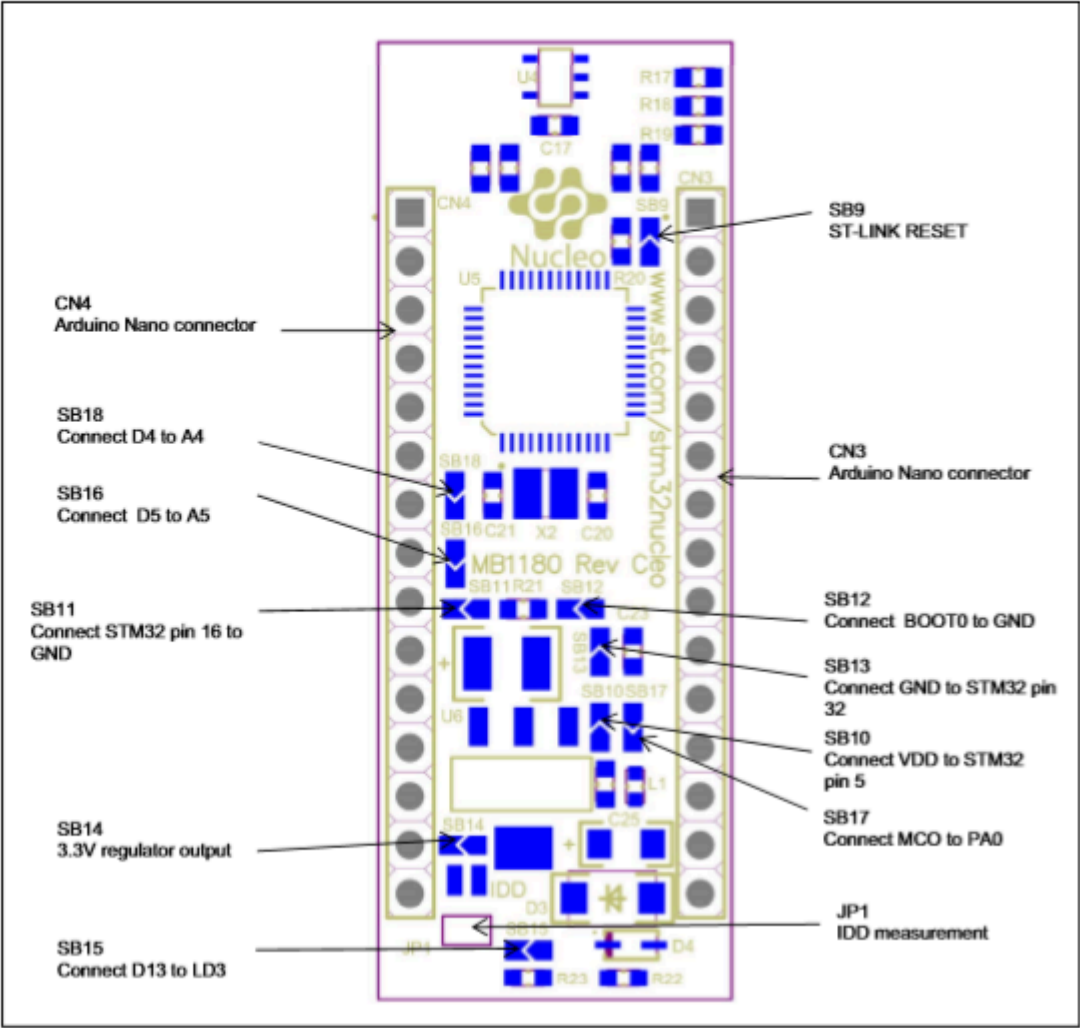


Figure 4. STM32 Nucleo-32 board bottom layout



顺带一提，ST不少官方板卡的晶振连接都不大对劲，有时同一板卡前后型号的晶振也有区别，大家使用时一定要多多核对。（我好像已经不是一次被坑了）。

再次提醒自己，为了避免各种匪夷所思、怀疑人生的情况出现，MCU出Bug时一定要先用示波器（物理）检查时钟。

后记

这是鄙人第一篇小记，还请各位看官多多指教~。